



YISHUN INNOVA JUNIOR COLLEGE
JC 1 PROMOTIONAL EXAMINATION
Higher 2

CANDIDATE
NAME

CG

INDEX NO

COMPUTING

Paper 2 Practical

9569/02

28 Sep 2023

1 hour 50 minutes

Additional Materials:

Removable storage device with the following files:

- ☐ PROMO_template.ipynb
- ☐ RIDERSHIP.txt
- ☐ VEHICLES.txt

READ THESE INSTRUCTIONS FIRST

Answer **all** questions.

All tasks must be done in the computer laboratory. You are not allowed to bring in or take out any pieces of work or materials on paper or electronic media or in any other form.

Approved calculators are allowed.

Save each task as it is completed.

The use of built-in functions, where appropriate, is allowed for this paper unless stated otherwise.

The number of marks is given in brackets [] at the end of each task.

The total number of marks for this paper is **60**.

Instruction to candidates:

Your program code and output for all the questions should be saved in a single file and downloaded as

PROMO_<your class>_<your name>.ipynb

For each of the sub-tasks, add a comment statement, at the beginning of the code using the hash symbol '#', to indicate the sub-task the program code belongs to, for example:

In [1]:

```
#Task 1.1  
Program Code
```

Output:

In [2]:

```
#Task 1.2  
Program Code
```

Output:

In [3]:

```
#Task 1.3  
Program Code
```

Output:

- 1 The solutions for the quadratic equation $ax^2 + bx + c = 0$ can be found using the following formula:

$$x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$

where $b^2 - 4ac$ is known as the discriminant.

Task 1.1

Write program code for the function `discriminant(a,b,c)` to calculate the discriminant for the quadratic equation $ax^2 + bx + c = 0$.

The function should return the discriminant if it is greater than or equal to zero. Otherwise, return `None`.

Test your program code with 3 test cases.

[3]

Task 1.2

Write program code for the function `quadratic_roots(a,b,c)` to calculate the solutions for the quadratic equation using the function `discriminant()` written in Task 1.1.

The function should provide the following outputs:

- ☐ `'No real root.'` if there is **no real root**
- ☐ the solution, correct to 1 decimal place, if there is a **repeated real root**
- ☐ a tuple containing the two solutions, both correct to 1 decimal place, if there are **two distinct real roots**.

For example:

<code>>>> quadratic_roots(2, 1, 2)</code>	Output: <code>'No real root.'</code>	
<code>>>> quadratic_roots(2, 4, 2)</code>	Output: <code>-1.0</code>	
<code>>>> quadratic_roots (2, 3, -2)</code>	Output: <code>(0.5, -2.0)</code>	[4]

2 Task 2.1

Write program code for the recursive function `oddiffy(lst)` to return a new list containing only the odd integers found in the list `lst`.

For example:

```
>>> oddify([1, 2, 3, 4, 5, 6, 7, 8])
```

```
Output: [1, 3, 5, 7]
```

[4]

Task 2.2

Write program code for the recursive function `str_to_int(string)` to return the value stated in the number string as an integer type.

For example:

```
>>> str_to_int('0012345')
```

```
Output: 12345
```

```
>>> str_to_int('1234500')
```

```
Output: 1234500
```

Do not use Python's built-in function `eval()`.

[4]

- 3 The Land Transport Authority of Singapore keeps track of the average daily ridership of our public transports, MRT, LRT, bus, and taxi. The text file `RIDERSHIP.txt` contains the data¹ for 1995 to 2016.

Task 3.1

Write program code to read the data from the text file `RIDERSHIP.txt` and store it in a list `ridership_list`.

An example of the list is as follows:

```
>>> ridership_list
```

```
Output: [['1995', 'MRT', '740000'],
         ['1995', 'Bus', '3009000'],
         ...
         ['2016', 'LRT', '180000'],
         ['2016', 'Bus', '3939000'],
         ['2016', 'Taxi', '954000']]
```

[5]

A researcher wants to compute the total and average daily ridership for any particular mode of public transport from 1995 to 2016 (inclusive).

Task 3.2

Write program code for the function `total(lst, tpt)` to compute using the data in the list `lst` and return the total daily ridership for a particular mode of transport `tpt`.

For example:

```
>>> total(ridership_list, 'MRT')
```

Output: 36057000

[3]

Task 3.3

Write program code for the function `average(lst, tpt)` to compute using the data in the list `lst` and the function `total(lst, tpt)` written in **Task 3.2** to return the average daily ridership for a particular mode of transport `tpt`.

For example:

```
>>> average(ridership_list, 'Taxi')
```

Output: 923625

[2]

¹ Contains information from *Public Transport Utilisation - Average Daily Public Transport Ridership*, accessed on 24 Aug 2023 from Land Transport Authority which is made available under the terms of the Singapore Open Data Licence version 1.0 <https://beta.data.gov.sg/open-data-license>

- 4 A programmer wrote the program code for a computer game where a player collects gems while navigating through a maze in different stages.

The programmer initialised an empty Python dictionary, `bag`, to store the player's collections:

```
bag = {'diamond':0, 'emerald':0, 'ruby':0, 'sapphire':0}
```

In Stage One, the player collected a diamond and a ruby.

Task 4.1

Write program code to update `bag` for the diamond and ruby collected in Stage One. [2]

In Stage Two, the sequence of gems collected by the player is recorded in the following list,

```
seq = ['emerald', 'diamond', 'ruby', 'sapphire', 'emerald']
```

Task 4.2

Write program code to iterate through the list `seq` and update `bag` for the gems found in Stage Two. [3]

In Stage Three, the player sells the gems in exchange for gold coins.

The exchange rates for the gems are as follows:

```
rate = {'diamond':40, 'emerald':35, 'ruby':30, 'sapphire':20}
```

Task 4.3

Write program code to:

- ☐ initialise a new key `'coin'` in the Python dictionary `bag`,
- ☐ iterate through the `bag` to exchange all the gems into coins, and
- ☐ update the items in `bag`.

[5]

- 5 For a class of 20 students participating in the 2.4km NAPFA run, the time taken (in minutes) by each student to complete the run is recorded in the following list:

```
timings = [ 11.5, 15.8, 10.3, 13.0, 12.1, 9.5, 14.8,  
            11.2, 10.0, 13.7, 11.0, 12.4, 17.6, 12.0,  
            12.5, 11.8, 9.9, 10.2, 14.3, 12.3 ]
```

The following is an implementation of the Merge Sort algorithm that operates on an array seq:

```
FUNCTION MergeSort(seq)  
    //Base case  
    IF length(seq) <= 1 THEN  
        RETURN seq  
    ENDIF  
  
    //Divide the list into two halves  
    mid ← length(seq) // 2  
    left ← _____<A>_____  
    right ← _____<B>_____  
  
    //Recursively sort both halves  
    left ← _____<C>_____  
    right ← _____<D>_____  
  
    //Merge the sorted halves  
    result ← Merge(left,right)  
    RETURN result  
ENDFUNCTION
```

FUNCTION Merge(left,right)

result ← an empty list

WHILE left is not empty AND right is not empty

IF first item of left ≤ first item of right THEN

_____ <E> _____

ELSE

_____ <F> _____

ENDIF

ENDWHILE

IF left is empty

_____ <G> _____

ELSE

_____ <H> _____

ENDIF

RETURN result

ENDFUNCTION

Task 5.1

Write the pseudocode for **A** to **H** in the functions.

[4]

Task 5.2

Write program code for the functions MergeSort(seq) and Merge(left, right) to implement the above Merge Sort algorithm.

Test your program code with the data timings.

[3]

The timings for the students are stored in another list containing tuples (name, timing):

```
class_data = [ ('ELAINE', '12.5'), ('GEMMA', '14.3'),  
               ('JACK', '12.0'), ('OLIVE', '11.5'),  
               ('QUINN', '10.2'), ('CASSIE', '9.5'),  
               ('ALEX', '13.7'), ('PETER', '12.1'),  
               ('TOM', '11.2'), ('MIA', '12.3'),  
               ('ROSE', '14.8'), ('BEN', '11.0'),  
               ('DAN', '12.4'), ('FRED', '9.9'),  
               ('LEE', '17.6'), ('HARRY', '11.8'),  
               ('IRIS', '10.0'), ('KELLY', '15.8'),  
               ('NICK', '10.3'), ('SARA', '13.0') ]
```

Task 5.3

Write a program code `sorting(data)` to return a list of tuples with the timings arranged in an increasing order.

Do not use any Python's built-in sorting function.

Test your program code with the students' timings `class_data`.

[3]

- 6 A vehicle repair depot accepts all types of vehicles for maintenance and repair. The Customer Service Department uses a hash table to keep track of the locations of the vehicles and the status of their repairs.

A hash value is generated using the vehicle's registered number.

An example of the hash value is produced as follows:

- ☐ vehicle's number plate: SBK4125T
- ☐ a set of products is calculated as shown:

Vehicle number plate	ASCII code	Multiplier	Product
S	83	8	664
B	66	7	462
K	75	6	450
4	52	5	260
1	49	4	196
2	50	3	150
5	53	2	106
T	84	1	84
Total			2372

- ☐ the total 2372 is then divided by 100 to give remainder 72, which is the index in the hash table that stores the vehicle data.

Task 6.1

Write program code for the function `hash(num)` to return the hash value for the vehicle registered number `num`. [3]

The vehicle data in the text file `VEHICLES.txt` contains the vehicle's registered number, the workshop, the date, and the status of the repair.

Task 6.2

Write program code to:

- ☐ create a hash table `data_table` of size 100
- ☐ read all the vehicles' data from the text file `VEHICLES.txt`
- ☐ use the `hash(num)` function in **Task 6.1** to generate the indices to store the vehicles data in the hash table `data_table` [6]

Task 6.3

Write program code for the function `query(num)` to return the status of the vehicle so that the customer service officer can use it to address the customer's enquiry

For example:

```
>>> query('STU3456T')
```

Output: 'Pending for repair' [4]

Task 6.4

Write program code for the function `ready(num)` which allows the mechanics to update the status of the vehicle to 'Ready for collection' and output the statement 'Data updated'.

```
>>> ready('XYZ9012Y')
```

Output: 'Data updated' [2]

END OF PAPER

BLANK PAGE